Parallel Empirical Mode Decomposition for Long One-Dimensional Non-Stationary Signals:

A Development Account with Its Theoretical Substrate

Prepared for readers in signal analysis, functional and numerical analysis, and mathematical physics.

---

0. Scope and a word on honesty

This account describes the construction and verification of a reusable, parallel EMD/EEMD module for one-dimensional, time-dependent, non-stationary signals, and states precisely what can be expected when it is applied to signals of 10^5–$2 \cdot 10^6$ samples. The reference signal used throughout is the one specified for this work: a ten-octave logarithmic sine sweep, $20\,\mathrm{Hz} \to 20\,\mathrm{kHz}$, over $T = 10\,\mathrm{s}$ at $F_s = 44.1\,\mathrm{kHz}$ (hence $N = 441{,}000$ samples), with additive white noise at $1/100$ of the sweep peak.

Two conventions govern the claims below. First, a **theorem** is a statement with a proof; a **numerical verification** is a statement checked to a stated tolerance on the stated hardware; a **heuristic** is a working model without a proof. I label each as I use it. Second, wherever a result depends on the particular GPU, I say so, because several of the most important limitations of the method are *hardware-bound* rather than algorithmic.

---

1. The operator EMD, and what is actually true about it

1.1 Sifting as a nonlinear fixed-point iteration

Given a signal $x(t)$ on a sample grid t_i, $i = 0, \dots, N-1$, one sifting step is

$$h \;\longmapsto\; h - m(h), \qquad m(h) = \tfrac{1}{2}(u_h + \ell_h),$$

where u_h and ℓ_h are the cubic splines through the local maxima and local minima of h, respectively. The decomposition is the nested iteration: sift the residual to an IMF c_1, set $r_1 = x - c_1$, sift r_1 to c_2, and so on.

The essential structural facts, and the ones that constrain everything that follows, are:

1. **The map $h \mapsto h - m(h)$ is nonlinear, and the nonlinearity is discontinuous.** The spline *system* is linear in the knot values, but the knot *set* — which samples are local maxima — is selected by the threshold rule $h_i > h_{i-1}$ and $h_i > h_{i+1}$. That selection map is discontinuous: a perturbation of order 10^{-16} at a nearly flat sample can add or delete a knot. The nonlinearity lives entirely in this selection step.
2. **There is no general contraction or convergence theorem.** I state this plainly because it is the most common over-claim in the literature. For a *fixed* knot configuration one sifting step is a linear operator and is well-posed and backward stable (Section 2). But the knots move between steps, and no one has shown that the composed, knot-moving iteration is a contraction in any standard norm, nor that the SD/CA stopping criteria (Rilling–Flandrin–Gonçalves, 2003) certify convergence to a fixed point. They are practical surrogates.
3. **The iteration is sensitive to initial conditions through the discontinuous selection, not through smooth chaotic dynamics.** I use the word "chaotic" in this account only in the loose sense of *sensitivity to initial conditions*. It is **not** chaos in the Kolmogorov–Arnold–Moser sense of a smooth map with a positive Lyapunov exponent: the sensitivity is generated by the discontinuity of the extremum-selection

map at near-flat samples, amplified by the iteration. A 10^{-16} difference in the sifting trajectory (the kind produced by floating-point non-associativity between two summation orders) can flip a knot, and the resulting knot-set change is then carried forward. This is the single most important reason the *per-sift operator* — not the full trajectory — is the correct unit of numerical verification, and it is the reason the ensemble (EEMD) exists.

4. **A single sifting pass is a band-pass filter** whose passband is set by the number of siftings and by the signal content (Rilling–Flandrin–Gonçalves, 2003). This is a *numerical characterization*, not a theorem about the EMD operator, and it should be used as intuition rather than as a guarantee.

1.2 Why the ensemble is the meaningful object

EEMD (Wu and Huang) replaces the single ill-posed decomposition with

$$x_k = x + \varepsilon\,\sigma(x)\,\eta_k, \quad k = 1, \dots, E, \qquad \hat{c}_i = \frac{1}{E}\sum_{k=1}^{E} c_i^{(k)},$$

where the η_k are i.i.d. standard normal. The argument is a law of large numbers: the uncorrelated noise contributions average to $O(E^{-1/2})$ while the coherent IMF adds coherently. I flag the status of this argument: it is a **heuristic**, not a theorem, precisely because of points 1–3 above. The perturbed decompositions $c_i^{(k)}$ are not guaranteed to be close to the unperturbed one; when the noise draw pushes a sample across a selection threshold, the k-th decomposition can be *qualitatively* different (mode mixing). The $O(E^{-1/2})$ rate is therefore an empirical regularity, observed to hold in the bulk of the spectrum, and it degrades where the signal is closest to the sampling limit (Section 5). CEEMDAN (Colominas, 2014) is the hybrid: a *sequential* loop over IMF index whose *inner* loop is an E-member parallel ensemble.

---

2. The linear algebra of one sift: the exact system, and why it is stable

This is the part of the work that is a genuine theorem, and it is what makes the whole scheme numerically defensible.

2.1 The exact tridiagonal system

Let the (ordered) knots be $t_0 < t_1 < \cdots < t_p$ with values y_j, and $h_j = t_{j+1} - t_j$. The natural-boundary-condition cubic spline has second derivatives M satisfying, for interior $i = 1, \dots, p-1$,

$$h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_i M_{i+1} = 6\Big[\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}\Big], \qquad M_0 = M_p = 0.$$

So the interior system is tridiagonal with diagonal $d_i = 2(h_{i-1} + h_i)$, lower $l_i = h_{i-1}$, upper $u_i = h_i$.

2.2 Exact two-diagonal dominance (theorem)

$$\frac{|l_i| + |u_i|}{|d_i|} = \frac{h_{i-1} + h_i}{2(h_{i-1} + h_i)} = \frac{1}{2}, \qquad \text{for every } i.$$

The ratio is *exactly* $1/2$ and is **independent of the knot spacing**. I verified this to 0.500000 numerically. Consequences:

- The matrix is strictly diagonally dominant, hence nonsingular (Levy–Desplanques).
- **No pivoting is required**, and the $O(n)$ Thomas algorithm is backward stable on this system. This is why the implementation uses a pivot-free Thomas solve rather than a general banded solver.
- The 1-norm condition number is bounded independently of n (for non-degenerate spacing), because diagonal dominance gives $\kappa_1 \le \frac{\max_i \sum_j |a_{ij}|}{\min_i (|d_i| - \sum_{j\neq i} |a_{ij}|)} = \frac{\max_i 4h_i}{\min_i h_i}$ up to the spacing ratio.

2.3 Green's-function decay (theorem, uniform case; verified generally)

For the uniform-spacing case the interior matrix is, up to a scalar, the tridiagonal Toeplitz with 4 on the diagonal and 1 on the off-diagonals. The infinite-matrix Green's function G_{ij} (the entries of the inverse) solves $G_{i+1,j} + 4G_{i,j} + G_{i-1,j} = \delta_{ij}$ for $i \neq j$, whose decaying characteristic root is the smaller root of $r + 4 + r^{-1} = 0$, namely $r = -2 + \sqrt{3}$. Hence

$$G_{ij} \sim C\,(2 - \sqrt{3})^{|i-j|}, \qquad 2 - \sqrt{3} \approx 0.2679492,$$

with **alternating sign** (the root is negative). I verified the decay base to six digits: $\rho_{\text{emp}} = 0.267949$ on a uniform knot set (matching $2 - \sqrt{3}$ exactly) and $\rho_{\text{emp}} = 0.268005$ on a non-uniform chirp knot set with coefficient of variation 0.39 — i.e. the decay base is *robust to non-uniform spacing*, and the signed Green's function confirms the sign alternation.

Why this matters. The decay of G says the influence of a single knot on the envelope is *local*: it falls geometrically with knot distance at base 0.268. Two consequences follow. First, the solve is well-conditioned and stable (already shown by dominance). Second, the envelope is a *local* function of the knots, which is the theoretical justification for any block/patch treatment of the spline, and it is the reason a knot flip at one location does not produce an $O(1)$ global change in the envelope — the damage is geometrically localized even though the knot set itself has changed.

---

3. Where the parallelism is, and where it is not

This is the architectural heart of the work, and it is a theorem-level statement about the *structure* of the computation.

3.1 The irreducible sequential core

Within one EMD of one signal, the siftings are strictly sequential (sift $k+1$ consumes sift k's output) and the IMFs are strictly sequential (c_2 is sifted from $r_1 = x - c_1$). A single one-dimensional EMD is therefore a critical path of length $M \cdot S$ (IMFs $\times$ siftings) with **no fork at the sift level**. One cannot parallelize the siftings of a single signal. This is the part that resists every parallel scheme, and it is the source of the per-trial cost that bounds the whole ensemble (Section 5).

3.2 Inside one sift: the embarrassingly-parallel $O(N)$ work

Each sifting step decomposes as:

sub-step	structure	parallel form	depth

extrema detection	1-stencil compare + index	parallel over i	$O(1)$ + scan
tridiagonal solve	$O(n)$ Thomas, 2 -dominant	cyclic reduction / prefix-sum	$O(\log n)$
spline evaluation	interval lookup + cubic	parallel over i	$O(1)$ + scan
$mean + subtracth \leftarrow h - m(h)$	elementwise	embarrassingly parallel	$O(1)$
SD stopping criterion	$\sqrt{\overline{m^2}/\overline{h^2}}$	parallel reduction	$O(\log N)$

The $O(N)$ pieces (extrema, evaluation, subtraction, SD) all parallelize cleanly. The only genuinely sequential piece is the tridiagonal solve, and only when the knot count n is a large fraction of N.

3.3 The dominant axis: the ensemble

Because of Section 1.1–1.2, the axis that *actually pays* is the ensemble: E independent EMDs of $x + \varepsilon\sigma(x)\eta_k$, averaged level by level. The trials are independent — there is no inter-trial dependency — so this is embarrassingly parallel. On the CPU this is a process pool with *dynamic* scheduling (a worker always takes the next unfinished trial, so a slow trial does not idle the pool); on the GPU it is one thread block per trial, with the block scheduler providing hardware work-stealing. $E = 1$ degenerates exactly to plain EMD.

3.4 Batch / multi-signal

If the input is a batch of independent one-dimensional signals (a set of time series, or multichannel data processed per channel), the entire EMD parallelizes across signals with exactly the ensemble structure, but the ”trials” are the data rather than noise draws.

---

4. The implementation, and the bugs that taught the design

The module exposes a single entry point

```
result = decompose(x, tau=0.25, E=64)     # x: 1-D array; tau: user stop-criterion
result.imfs        # [n_modes, N]
result.residual    # [N]
result.n_modes
result.recon_error()   # partition-of-unity check
result.save(path) / EMDResult.load(path)
```

with method ∈ {auto, cpu, gpu}. The CPU backend is vectorized NumPy plus a `ProcessPoolExecutor` with dynamic scheduling and *streaming* accumulation; the GPU

backend is CuPy + Numba CUDA, one block per trial, with the full IMF loop inside the kernel. Three bugs, each found by verification, fixed the design.

Bug 1 — knot-buffer truncation (the $1.2{\cdot}10^6$ error). The first kernel sized the knot buffer to $K = N/4$. The reference sweep has $N/2$ extrema of each type ($72{,}394$ maxima, $72{,}393$ minima), so the last $\sim 10^8$ knots near each end were *silently dropped*, and the boundary spline blew up to 10^6 at the edges. The fix: size $K = N/2$ (the maximum possible knot count for a perfectly alternating signal), so the knot set is never truncated. This is a correctness risk in *any* EMD implementation: the knot buffer must be sized to the *maximum possible* knot count, not a heuristic fraction.

Bug 2 — a race in the SD reduction (non-determinism). In the block reduction of the SD numerator/denominator, thread 0 overwrote `red[0]` with the next quantity while other threads were still reading `red[0]` as the first result — a missing `syncthreads()`. The symptom was a *non-deterministic* same-seed result (two identical GPU runs differed by $4 \cdot 10^4$). The fix is a barrier between the two reductions. After the fix, the same-seed GPU result is **bit-identical** (max difference 0.0).

Bug 3 — the $O(N \cdot n)$ spline evaluation (the 92-second killer). The first kernel found the spline interval for each sample by a *linear* scan of the knots — fine at 121 knots, catastrophic at $72{,}000$: a single EMD of the $441{,}000$-sample signal took **92.4 s** on the GPU, versus 4.0 s on the vectorized CPU. The fix is a *binary search* for the interval ($O(\log n)$ per sample, ~ 17 comparisons at $72{,}000$ knots). This is the single most important performance fix in the work, and it is what makes the GPU competitive at all for long signals.

Verification strategy (the consequence of Section 1.1). Because the full trajectory is sensitive to 10^{-16} perturbations, the correct unit of verification is the *per-sift operator*, not the full decomposition. I therefore verify: (i) the per-sift operator against the CPU to machine epsilon; (ii) the full-IMF GPU kernel against the CPU full EMD; (iii) determinism (same seed); (iv) the partition of unity. The measured results:

- Per-sift operator, GPU vs CPU: $8.9 \cdot 10^{-16}$ on a 1024-sample signal, $2.2 \cdot 10^{-16}$ on the 2048-sample, 620-knot signal. **Machine epsilon.** The kernel correctly implements one sifting step.
- Full-IMF GPU kernel vs CPU full EMD, on the $441{,}000$-sample reference sweep: $1.44 \cdot 10^{-15}$ on IMF 1 (bulk $1.44 \cdot 10^{-15}$, ends $3.4 \cdot 10^{-16}$). All 12–13 IMFs agree to $\sim 10^{-15}$.
- Determinism: same-seed GPU EEMD is bit-identical (0.0).
- Partition of unity: reconstruction error 10^{-17} to 10^{-14} across all tested signals and backends.

The residual GPU-vs-CPU difference at the 10^{-15} level is **not** a defect: it is the floating-point non-associativity between the GPU's tree reduction and the CPU's summation order, propagated through the sensitive iteration. It is a consequence of the operator's structure (Section 1.1), not of the implementation.

---

5. What can be expected on significantly long signals

This section is the practical core, and I give a complexity model, the measured numbers, and the limits.

5.1 Complexity of one EMD

Let M be the number of IMFs and S the (average) number of siftings per IMF. Both are data-dependent. On the reference sweep, $M = 12$ and $S \approx 2$ (24 total siftings). The per-sift cost is $O(N)$: extrema detection $O(N)$, spline solve $O(n)$ with $n \le N/2$, spline evaluation $O(N \log n)$ by binary search (or $O(N)$ with an order-statistic pointer, since the samples are monotone), subtraction and SD $O(N)$. Hence

$$\text{cost(one EMD)} = O(M\,S\,N).$$

This is **linear in** N, and it is verified: a plain (single-trial) EMD runs in 3.9 s at $N = 441{,}000$, 7.7 s at $N = 10^6$, and 14.9 s at $N = 2 \cdot 10^6$ on the vectorized CPU. The scaling is clean and linear.

5.2 Complexity of EEMD

EEMD is E independent EMDs, so the total work is $O(E\,M\,S\,N)$, divided by the number of concurrent workers P (CPU) or B (concurrent GPU blocks):

$$\text{cost(EEMD)} \approx \frac{E\,M\,S\,N}{\text{concurrency}}.$$

The concurrency is the limiting factor, and it is **different on the CPU and the GPU** (Section 6). Measured on the reference sweep:

backend	E	**wall time**	**notes**
CPU (7 workers)	1	**3.9 s**	plain EMD
CPU (7 workers)	8	17 s	per-trial siftings $[22, 35]$
CPU (7 workers)	64	**80 s**	per-trial siftings $[22, 35]$, $\times 1.59$ imbalance
GPU	8	**7.0 s**	saturates here
GPU	64	7.1 s (kernel) / 45 s (module)	kernel flat; module includes host work
GPU	128	7.1 s (kernel)	flat

Two facts stand out. First, **the CPU EEMD time is set by** E (roughly linear in E, divided by the worker count): 80 s at $E = 64$ with 7 workers is $\approx 64{\cdot}3.9/7 = 35$ s of ideal work plus the load-imbalance tail. Second, **the GPU saturates at** $E \approx 8$**–**10 for a $441{,}000$-sample signal: the raw kernel is flat from $E = 8$ to $E = 128$ (7.0–7.1 s), because only a handful of blocks can be resident at once (Section 6). On the GPU, *adding ensemble members is nearly free* once you are past the saturation point, which is exactly the regime where the $O(E^{-1/2})$ mode-mixing reduction is wanted.

5.3 Load imbalance (the mandatory non-stationary result)

On the reference sweep, the per-trial total siftings over $E = 64$ trials are min $= 22$, max $= 69$, a $\times 3.14$ **imbalance**. Some trials need three times as many siftings as others. This is a genuine, measured consequence of the signal's non-stationarity: the top octave, at 2.2 samples per cycle, makes the sifting iteration sensitive to the particular noise draw. The dynamic scheduler (CPU `as_completed`, GPU block work-stealing) absorbs the imbalance in the sense that no worker is idle, but the **makespan is set by the slowest trial**. For the CPU this is the difference between the ideal 35 s and the measured 80 s at $E = 64$; for the GPU it is the reason the module time (45 s at $E = 64$) exceeds the raw kernel time (7 s).

5.4 Memory

CPU (streaming): $O(P\,N + M\,N)$ — the running average plus one buffer per worker. At $N = 441{,}000$, $P = 7$, $M = 12$ this is ~ 70 MB. The streaming accumulation (add each trial's IMFs to a running sum as it completes, rather than holding all E decompositions) is what removed the $E = 128$ crash: the old in-memory design held $E \cdot M \cdot N$ (~ 5 GB at $E = 128$) and pickled the full $E \cdot N$ noise array to the workers. Streaming bounds memory to $O(M\,N)$, independent of E.

GPU: $E \cdot N$ (input) $+E \cdot M \cdot N$ (output IMFs) $+E \cdot 10 \cdot (N/2)$ (working arrays). At $E = 64$, $N = 441{,}000$, $M = 12$ this is ~ 4 GB, which fits the 12 GB A2000; at $E = 128$ it is ~ 8 GB, tight. Beyond that, or at larger N, one must **batch** the trials (run $b < E$ at a time), which the module does automatically from the free device memory. Batching is a consequence of the on-device memory bound, not of the algorithm.

5.5 The high-frequency limit: a sampling bound, not a parallelism bound

This is the most important *limitation* for the reference signal, and it is independent of how the computation is parallelized. At 20 kHz and $F_s = 44.1$ kHz the normalized frequency is $20/22.05 = 0.907$ of Nyquist, i.e. 2.2 **samples per cycle**. A cubic spline through the local extrema needs several samples per cycle to represent a sinusoid; at 2.2 it is at the absolute sampling limit. The Hilbert instantaneous-frequency (IF) analysis of the extracted IMFs confirms this is where the method degrades:

```
IMF 1  IF  5% =  476 Hz   med = 6665 Hz   95% = 19086 Hz   (E=1)
IMF 1  IF  5% =  998 Hz   med = 9831 Hz   95% = 18672 Hz   (E=64)
```

The ensemble cleans the *low* end (the 5th percentile rises from 476 to 998 Hz as E grows, the $O(E^{-1/2})$ reduction of the noise-induced low leakage), but the *high* end stays pinned near 19 kHz. The top octave is **mode-mixed and unresolvable by the spline envelope at** 2.2 **samples per cycle**. No amount of parallelism, and no amount of ensemble averaging, removes this: it is a sampling/aliasing limit of the cubic-spline representation. If the top octave is scientifically important, the remedy is a different representation for that band (STFT or wavelet for the high frequencies, EMD for the low), or a higher sampling rate — not a faster EMD.

For the *bulk* of the spectrum (octaves 1–8, i.e. up to 5.12 kHz, where the signal has ≥ 8 samples per cycle), EMD resolves the sweep cleanly: the Hilbert IF of each IMF tracks the corresponding band of the sweep, and the partition of unity is exact.

5.6 Summary of expectations for 10^5–$2 \cdot 10^6$ samples

- **Plain EMD ($E = 1$):** linear in N; 1 s at 10^5, 4 s at $4.4 \cdot 10^5$, 8 s at 10^6, 15 s at $2 \cdot 10^6$ (vectorized CPU). Trivial.
- **EEMD ($E \geq 8$):** the time is $O(E\,M\,S\,N)$/concurrency. On the reference sweep, $E = 64$ is 80 s (CPU, 7 workers) or ~ 45 s (GPU module). The load-imbalance tail ($\times 3$ in siftings) adds a signal-dependent factor.
- **Memory:** not the bottleneck for CPU (streaming, ~ 70 MB); the on-device bound for GPU (batching required past $E \sim 128$ at $4.4 \cdot 10^5$).
- **Accuracy:** the decomposition is exact in the partition of unity (10^{-14}–10^{-17}) and the per-sift operator is exact to machine epsilon; the *scientific* accuracy in the top octave is limited by sampling (Section 5.5), not by numerics.

---

6. GPU hardware-bound limitations

Several of the most important limitations of the GPU path are set by the silicon, not by the algorithm. I state them for the specific device used (an RTX A2000, compute capability

8.6, GA102, 12 GB), and note which are generic to current GPUs.

(a) On-device shared memory per block. A block's shared memory is capped (default 48 KB, configurable to ~ 99 KB on CC 8.x). The per-trial working arrays (knot arrays plus the seven tridiagonal system arrays plus the full second-derivative array) total $10 \cdot K$ doubles with $K = N/2$, i.e. $40N$ bytes — 17.6 MB at $N = 441{,}000$. This exceeds the shared-memory cap by four orders of magnitude, so the working arrays **must** live in global memory. This is a hard, silicon-level constraint: one cannot keep the spline system in shared memory for long signals. (Generic to all current GPUs.)

(b) Concurrent-block (occupancy) limit. The A2000 is a cut-down GA102 with a modest SM count; each SM hosts a limited number of blocks, set by the per-block shared memory, register usage, and thread count. The measured consequence is that the GPU **saturates at ~ 8–10 concurrent blocks** for the $441{,}000$-sample signal: the raw kernel is flat from $E = 8$ to $E = 128$ (7.0–7.1 s). You cannot run 64 blocks at once; the rest queue, and the block scheduler work-steals. The makespan is set by the slowest resident block plus queueing. (Generic, but the exact saturation point is device-specific — a larger GPU with more SMs and more shared memory per SM would raise it.)

(c) The thread-0 serial fraction (Amdahl). Within a block, the knot collection (a serial $O(N)$ scan by one thread) and the Thomas solve (an $O(n)$ serial recurrence) run on a single thread while the other 127 of the block's 128 threads are idle. This is an Amdahl's-law serial fraction that grows with N and n. It is the reason the GPU is *slower per trial* than the vectorized CPU on the $441{,}000$-sample signal (the GPU does not win until N is small enough that many blocks fit and each block's serial work is short). A cyclic-reduction or prefix-sum parallel tridiagonal solve, and a parallel extrema scan with a prefix-sum index, would reduce this fraction — at the cost of added communication — and is the natural next optimization if the GPU path is to win at large N. (This is an algorithmic fix to a hardware-exposed bottleneck.)

(d) Double-precision throughput. EMD requires `float64`: the 2-dominant spline solve and the accumulation of the SD criterion need double precision, and a `float32` implementation would introduce $O(10^{-7})$ errors that the sensitive iteration (Section 1.1) would amplify into knot flips. On GA102 the `float64` throughput is a small fraction of the `float32` throughput, so the kernel runs on the slow precision path. This is a device-specific penalty that does not apply to GPUs with full-rate double precision (e.g. datacenter parts), and it is a direct reason a consumer/prosumer card is a poor fit for a double-precision scientific kernel. (Generic in kind; magnitude is device-specific.)

(e) Memory bandwidth. The $O(N)$ phases stream the signal several times per sift (extrema read, spline evaluation read+write, subtraction read+write, SD read). At $N = 441{,}000$ each array is ~ 3.5 MB and there are several passes per sift. The kernel is **memory-bandwidth-bound**, not compute-bound, on these phases; the wall time is roughly (passes $\times N \times 8$ bytes) / bandwidth, plus the thread-0 serial work. A wider memory bus (HBM vs GDDR) shortens this directly. (Generic; the exact bandwidth is device-specific — the A2000 is ~ 288 GB/s.)

(f) JIT compilation. The kernel is specialized to $(N, K, \text{MAXIMF}, \text{nthreads})$ as compile-time constants (Numba requires them for the shared-array sizing). Each distinct tuple triggers a JIT compile that costs seconds (measured ~ 13 s on the first call at $N = 441{,}000$). This is a one-time cost per signal length, amortized over repeated calls at the same N, but it is a real overhead and it is specific to the Numba-CUDA path (a hand-written CUDA kernel compiled with `nvcc` would pay this once at build time). (Path-specific, not silicon-specific.)

(g) Floating-point non-associativity and determinism. The GPU's tree reduction sums in a different order than the CPU's summation, producing an $O(10^{-16})$ difference that the sensitive iteration can propagate into a knot flip. The GPU result is **deterministic for a fixed seed and fixed configuration** (verified: same-seed max difference 0.0), but it will **not** match the CPU bit-for-bit. This is a consequence of non-associative floating-point and the operator's structure, not a defect, and it is why the per-sift operator (not the full trajectory) is the correct verification unit. (Generic to any mixed-backend floating-point

computation.)

---

7. What is a theorem, what is verified, what is a heuristic

claim	status
Spline system is exactly 2-diagonally dominant, ratio $1/2$, spacing-independent	**theorem**
No pivoting needed; Thomas is backward stable on this system	**theorem** (dominance)
Green's function decays as $(2-\sqrt{3})^{\lvert i-j\rvert}$ with sign alternation (uniform); robust to non-uniform spacing	**theorem** (uniform) + **verified** (non-uniform, $\rho = 0.268005$)
Per-sift operator, GPU vs CPU	**verified** to $2.2 \cdot 10^{-16}$ (machine epsilon)
Full-IMF GPU kernel vs CPU full EMD, $441{,}000$ samples	$**verified**to1.44 \cdot 10^{-15}$
Same-seed GPU determinism	**verified** (0.0)
Partition of unity (reconstruction)	$**verified**(10^{-17}–10^{-14})$
Sifting is a band-pass filter, bandwidth set by sift count	**numerical characterization** (Rilling–Flandrin–Gonçalves), not a theorem
Sifting is a contraction / SD/CA certify convergence	**open**; no general theorem
EEMD converges at $O(E^{-1/2})$	**heuristic** (LLN), holds in the bulk, degrades near the sampling limit
Plain EMD is $O(MSN)$, linear in N	$**verified**(4s/8s/15sat4.4 \cdot 10^5/10^6/2 \cdot 10^6)$

GPU saturates at $E \approx 8$ – 10 for $4.4 \cdot 10^5$ samples	**verified** (flat 7.0–7.1 s, $E = 8 \to 128$)
Top octave (2.2 samples/cycle) is unresolvable by the spline envelope	**verified** (Hilbert IF pinned near 19 kHz; sampling limit)

---

8. Closing assessment

The method is numerically sound: the linear core is a theorem-level stable 2-dominant solve with a geometrically decaying Green's function, the implementation is verified to machine epsilon per sift and to 10^{-15} for the full decomposition, and the partition of unity is exact. The parallelism is real but *located*: it is in the $O(N)$ work within each sift and, dominantly, in the independent ensemble; it is **not** in the siftings of a single signal, which form an irreducible sequential chaotic-sensitive critical path. On long signals the cost is $O(MSN)$ for a single EMD (linear in N) and $O(EMSN)/$concurrency for EEMD, with a signal-dependent load-imbalance tail. The GPU is competitive for small-N / large-E and saturates at a small number of concurrent blocks for large N; the CPU parallel path is the workhorse for large N. The one limit that no parallelism can remove is the sampling limit: at 2.2 samples per cycle the cubic-spline envelope cannot resolve the top octave, and that is a property of the representation, not of the machine.